

AI 에이전트 UX 설계

사용자가 실제로 쓰고 싶은 인터랙티브 AI 제품 만들기

AI 에이전트 UX 설계

사용자가 실제로 쓰고 싶은 인터랙티브 AI 제품 만들기

ThakiCloud (Hyojung Han)

Contents

1	신뢰의 토대: 에이전트가 먼저 보여줘야 할 것	1
2	기대 관리: 무대화하지 않는 약속의 기술	5
3	실패 복구: 에이전트가 무너졌을 때의 설계	9
4	적응적 피드백 루프: 사용자와 에이전트의 공진화	13

1 신뢰의 토대: 에이전트가 먼저 보여줘야 할 것

AI 에이전트 제품을 처음 실행한 사용자는 무의식중에 같은 질문을 던진다. 이걸 믿어도 되는가. 이 물음에 대한 답은 처음 몇 초 안에 정해진다. 에이전트가 지금 무엇을 하고 있는지, 무엇을 할 수 있고 없는지를 빠르게 파악하지 못하면 사용자는 조작을 포기하거나 제품을 잘못 쓰거나 심한 경우 실제 손실을 입는다. 이 장은 에이전트가 첫 만남에서 신뢰의 토대를 쌓으려면 반드시 충족해야 할 설계 원칙을 다룬다.

1.1 상태 표시: 에이전트의 현재 위치를 밝히라

에이전트가 아무 말 없이 추론만 반복하면 사용자는 불안해진다. 무엇을 하고 있는지, 어디까지 끝냈는지, 왜 멈춰 있는지 모르는 채로 사용자는 둘 중 하나를 택한다. 초조하게 방해하거나 아예 손을 놓거나. 둘 다 생산적이지 않다.

좋은 상태 표시는 세 가지 축으로 구성된다. 작업 축에서는 현재 어떤 단계를 처리 중인지, 전체 작업에서 어느 위치에 있는지 백분율로 표시한다. 시간 축에서는 경과 시간과 예상 잔여 시간을 함께 보여준다. 다만 예상 시간은 정확하지 않을 때가 많으므로 “약 2분”처럼 범위로 표현하는 편이 현명하다. 오류 축에서는 실패가 발생했을 때 그것이 치명적인지 일시적인지를 명시적으로 구분한다.

구체적인 예로 여행 일정 에이전트를 생각해보자. 이 에이전트가 “파리 5일 일정을 작성 중...”이라고만 표시하면 사용자는 불안하다. 대신 “파리 5일 일정을 작성 중 (3/7단계: 숙소 검색 완료, 항공편 검색 중)”라고 표시하면 현재 위치를 직관적으로 파악할 수 있다. 여기에 예상 잔여 시간이 “약 1분

30초”로 표시되면 사용자는 기다림의 의미를 인지하고 다른 작업을 병행할 수 있다.

1.2 능력과 한계의 선제적 선언

에이전트가 할 수 있는 것과 없는 것을 미리 알리지 않으면 사용자는 뭐든 해줄 거라 기대했다가 실망한다. 이 간극을 메우는 가장 효과적인 방법은 에이전트의 행동 반경을 처음 만나는 순간 명확히 밝히는 것이다.

단순하게 “이 에이전트는 다음을 할 수 있습니다”와 “이 에이전트는 다음을 할 수 없습니다”를 한눈에 볼 수 있게 나열하는 방식이 있다. 목록은 짧고 구체적이어야 사용자가 실제로 읽는다. 나열이 길어지면 오히려 아무도 읽지 않는다.

좋은 예시로 일정 작성 에이전트를 생각해보자. 이 에이전트가 화면에 다음을 표시하면 사용자는 자신의 필요와 맞는지 곧바로 판단할 수 있다. “국내 항공편 검색과 예약, 숙소 검색과 예약, 렌터카 검색과 예약, 관광지 정보 제공”이라는 능력 목록과 “비행기 내 발권, 호텔 현장 결제, 비자 신청, 보험 가입”이라는 한계 목록이 함께 있으면 사용자는 에이전트의 적용 범위를 벗어나지 않는 요청만 하게 된다.

능력과 한계를 선언할 때 흔히 발생하는 오류는 너무 넓게 쓰거나 너무 좁게 쓰는 것이다. “다양한 작업을 수행할 수 있습니다”라는 표현은 사실상 공허하다. 구체적으로 열거해야 한다. “검색, 요약, 분석, 보고서 작성”처럼 실제 행동 동사를 사용한 목록이 좋다. 반대로 한계를 표현할 때 “기타 여러 기능”이라는 표현도 공허하다. “실시간 정보 조회, 타인 명의 예약, 결제 실패 시 자동 환불”처럼 구체적으로 열거한다.

1.3 진행 상황의 명시적 피드백

에이전트가 단계를 넘어갈 때마다 사용자에게 알리는 편이 좋다. 다만 사소한 변화까지 전부 보고하면 오히려 노이즈가 늘어 사용자가 정보 부족에 빠진다.

이때 균형을 잡아주는 것이 단계별 마커다. 작업이 여러 단계로 구성되어 있을 때 각 단계의 시작과 끝을 알려주면 사용자는 전체 진행률을 직관적으로 가늠할 수 있다. 예컨대 보고서 작성 에이전트가 “1) 자료 수집 완료 → 2) 자료 분석 중 → 3) 초안 작성 중 → 4) 초안 검토 중”처럼 진행 상황을 표시하면 사용자는 에이전트가 어떤 단계에 있는지 즉시 파악할 수 있다.

특히 중단과 재개가 가능하다는 사실은 자주 강조해도 좋다. “작업을 일시정지했습니다. 언제든지 이어서 진행할 수 있습니다”라는 메시지는 사용자에게 통제감을 준다. 에이전트가 백그라운드에서 작업을 계속하고 있다는 사실조차 알려주면 좋다. “이 작업을 백그라운드에서 계속 진행하며, 완료되면 알려드리겠습니다”라는 표현은 사용자를 불안으로부터 해방한다.

1.4 불확실성의 정직한 표현

에이전트가 확신하지 못하는 내용을 마치 확신하는 것처럼 표현하면 신뢰가 무너진다. “이 정보가 정확합니다”라고 했다가 틀리면 사용자는 에이전트를 처음부터 신뢰하지 않게 된다. 대신 “이 정보는 2024년 자료를 기반으로 추정했으며, 최신 자료와 다를 수 있습니다”라는 표현이 현실점에서는 더 적절하다.

구체적인 수치나 날짜일수록 불확실성이 크다. “추정 약 3일”이라는 표현은 “3일”보다 정확하다는 인상을 주지 않으면서도 사용자에게 불확실성을 암시한다. 이때 참조한 출처나 방법을 함께 밝히면 훨씬 신뢰할 만한 진술이 된다. “제 학습 데이터 기준(2024년 12월)으로 추정한 금액입니다.

실제 비용과 차이가 있을 수 있습니다”라는 문구가 있으면 사용자는 판단을 내릴 수 있다.

불확실성을 표현하는 빈도도 설계 대상이다. 매 순간 불확실성을 강조하면 사용자는 에이전트가 하는 모든 말에 의심의 눈길을 보내게 된다. 대신 결과의 신뢰도에 따라 등급을 나누면 효과적이다. 신뢰도가 높은 구간에는 평소처럼 답하고, 낮은 구간에만 “추정” 또는 “확신도 60%” 같은 태그를 붙이면 자연스럽게 구분된다.

1.5 이 장의 핵심 정리

신뢰는 첫 만남에서 정해지며 처음 몇 초가 이를 좌우한다. 에이전트가 지금 무엇을 하는지 명확히 보여주고 무엇을 할 수 있고 없는지 미리 선언하고 단계마다 진척 상황을 알리고 불확실성을 정직하게 표현하는 것, 이 네 가지가 신뢰의 기둥이다. 이를 충족하면 사용자는 에이전트를 도구로 받아들이고 그 도구를 더 넓게 쓸 준비를 한다.

2 기대 관리: 무대화하지 않는 약속의 기술

에이전트 제품을 쓸 때 사용자가 가장 크게 느끼는 좌절감은 기대와 현실 사이의 간극에서 온다. 에이전트가 해줄 것 같았던 일이 안 되면 실망한다. 이 챕터에서는 그 기대를 적절히 관리하기 위한 설계 기법을 다룬다.

2.1 행동 반경의 선제적 프레이밍

에이전트가 “할 수 있는 모든 것”을 나열하는 방식은 충분하지 않다. 사용자는 그 목록을 자신의 문제에 대입해서 생각하므로, 자신의 상황에서 에이전트가 구체적으로 무엇을 해줄 수 있는지 알아야 한다.

선제적 프레이밍의 핵심은 사용자가 지금 해결하려는 문제에 초점을 맞추는 것이다. 작업 시작 시 에이전트가 “무엇을 해드릴까요?”라고 묻기보다, “어떤 작업을 진행할까요?”라고 묻고 선택지에 따라 수행 가능한 범위를 보여주는 방식이 훨씬 효과적이다. 사용자가 “이메일 정리 좀 해줘”라고 의뢰하면, 에이전트는 “이메일 정리를 도와드리겠습니다. 다만 스팸 처리는 자동으로, 중요 메일 분류는 추천만 드리며, 실제 삭제는 확인 후 진행합니다”라는 식으로 범위를 명시한다.

이런 대화형 프레이밍의 장점은 사용자가 에이전트의 한계를 직접 부딪히며 배우는 대신 작업을 맡기기 전에 미리 파악할 수 있다는 점이다. 그러면 기대가 현실에 맞춰지고, 기대가 현실을 넘어서는 순간 발생하는 실망을 사전에 막을 수 있다.

2.2 과잉 약속의 메커니즘

에이전트를 만드는 입장에서 “네, 가능합니다”라고 대답하는 편이 대화상으로는 쉽다. 하지만 그 자리에서의 안이한 승낙은 나중에 신뢰를

훼손한다. 사용자가 “그것도 되나요?”라고 물을 때마다 “네!”라고 답하면, 실제로 되지 않는 기능까지 있는 것처럼 인상을 주게 된다.

과잉 약속은 세 가지 형태로 나타난다. 가장 흔한 것은 기능의 확실성을 과장하는 경우다. “자동으로 처리됩니다”라고 했는데 실제로는 반자동이면, 사용자는 매번 확인 과정을 거쳐야 하니 좌절감이 쌓인다. 두 번째는 처리 능력의 범위를 실제보다 넓게 해석하는 경우다. “데이터를 분석해드립니다”라는 말과 “대규모 데이터셋을 초 단위로 분석해드립니다”라는 말은 전혀 다른 수준이다. 마지막은 완료의 기준을 애매하게 표현하는 경우다. “완료되었습니다”라는 말은 에이전트가 생각하는 완료와 사용자가 생각하는 완료가 다를 수 있다는 문제를 안고 있다.

이 세 가지 문제는 구체적인 표현으로 막을 수 있다. “기본적으로 자동 처리되며, 일부 수동 확인이 필요할 수 있습니다”는 애매하지 않은 표현이다. “대규모 데이터셋은 처리 시간이 오래 걸릴 수 있으며, 10만 건 이상은 분할 처리로 진행합니다”는 능력의 한계를 구체적 수치와 함께 보여준다. “초안 작성 완료. 검토 후 수정이 필요하시면 말씀해 주세요”는 완료가 단계적이라는 사실을 암시한다.

2.3 단계별 목표 산출

복잡한 작업을 한 번에 약속하면 실패 확률이 높아진다. 대신 전체 작업을 논리적 단계로 나누고 각 단계의 산출물을 명시적으로 약속하면 사용자는 단계마다 성과물을 확인할 수 있어 불안이 줄어든다.

여행 일정 작성을 예로 들면, “파리 5일 일정표를 만들어 드리겠습니다”라는 말은 완성물을 약속하는 것이다. 하지만 그 결과물을 두고 사용자가 갖는 기대는 에이전트가 생각한 것과 다를 수 있다. 대신 “먼저 항공편을 검색해서 추천드립니다. 확인 후 숙소를 검색하고, 이어서 관광 코스를 제안하고, 마지막으로 전체 일정을 종합해서 정리해 드리겠습니다. 각

단계마다 확인을 요청하므로 필요하시면 수정하실 수 있습니다”라는 식으로 설명하면, 사용자는 전체 프로세스를 파악하고 각 단계에서 기대를 조정할 수 있다.

단계별 산출에는 또 하나의 장점이 있다. 실패의 범위를 한정할 수 있다는 점이다. 전체를 약속했다가 실패하면 전부 실패한 느낌을 주지만, 여러 단계로 나뉘면 성공한 단계는 그대로 유지되고 실패한 단계만 재시도하면 된다. 이는 정신적 부담을 줄이는 동시에 에이전트에 대한 신뢰도 부분적으로나마 지킬 수 있게 한다.

2.4 실패 가능성의 사전 경고

실패는 불가피하다. 실패를 사전에 전달해 사용자의 기대를 적절히 조절해주는 일이 관건이다. 그렇다고 모든 가능성을 나열하면 오히려 노이즈가 된다. 효과적인 사전 경고는 작업 맥락에 따라 발생 가능성이 높은 실패 유형에 집중한다.

데이터 분석 작업을 예로 들면, “분석을 시작하겠습니다. 다만 다음의 경우 결과가 부정확할 수 있습니다: 데이터에 결측치가 많은 경우, 과거 트렌드와 현 상황의 괴리가 큰 경우, 예측 기간이 너무 긴 경우”라는 식으로 주요 위험을 꼽아줄 수 있다. 사용자가 이를 미리 알고 있으면, 실제로 문제가 발생했을 때 “아, 그랬군요”라는 반응으로 충분하고 “왜 이렇게 결과가 엉뚱하지?”라는 반응은 나오지 않는다.

사전 경고의 타이밍도 중요하다. 작업 시작 시 전처리를 마치고 실제 분석 직전에 경고하는 방식이 가장 효과적이다. 분석을 시작하기 전이라면 방법론을 조정할 수 있지만, 이미 시작했으면 조정할 수 없기 때문이다. 분석 도중 예상하지 못한 문제가 발견되면, 즉시 사용자에게 보고하고 조언을 구하는 편이 결과만 나중에 전달하는 것보다 낫다.

2.5 이 장의 핵심 정리

기대 관리는 에이전트 설계에서 가장 많은 공을 들여야 하는 영역 중 하나다. 행동 반경을 선제적으로 프레이밍하고, 과잉 약속을 자제하며, 복잡한 작업은 단계별로 나눠 각 단계의 산출물을 약속하고, 실패 가능성은 미리 경고하는 설계가 핵심이다. 이 원칙을 따르면 사용자는 에이전트에 대해 정확한 기대를 갖고, 기대한 범위 안의 결과물에 만족하며, 실패가 발생해도 불안해하지 않고 대응할 수 있다.

3 실패 복구: 에이전트가 무너졌을 때의 설계

에이전트는 반드시 실패한다. 네트워크 오류, API 한도 초과, 처리 중 예외, 사용자의 잘못된 입력 등 실패의 원인은 다양하다. 문제는 실패 자체가 아니라 실패 이후의 대응이다. 사용자에게 제어권을 어떻게 돌려주고 실패를 어떻게 설명하며 어떤 재시도 경로를 제시하느냐가 제품의 완성도를 가른다.

3.1 부분 완료 상태의 유지와 제어권 반환

완전한 실패보다 부분 완성 후 실패가 더 위험하다. 사용자가 “이제 다 됐구나” 하고 다음 단계로 넘어간 뒤에 에이전트가 막판에 실패하면, 그때 드는 시간과 마음의 손실은 훨씬 크다. 이미 완료된 줄 알았던 작업의 결과를 확인하지 않은 채 건너뛰기 때문이다.

부분 완료 상태를 유지하는 설계는 간단하다. 에이전트가 작업을 진행하면서 이미 완성된 부분을 즉시 저장하고 시각적으로 보여주면 된다. “숙소 3곳을 찾았으나 항공편 검색에서 오류가 발생했습니다. 검색 결과를 저장했으니 오류 해결 후 이어서 진행 가능합니다”라는 메시지는 사용자에게 다음 행동을 명시한다. 저장된 결과를 보기 버튼과 함께 보여주면 사용자는 에이전트가 어느 정도까지 작업했는지 알고 있다는 사실에 안도한다.

제어권을 돌려준다는 것은 사용자에게 선택지 세 가지를 넘긴다는 뜻이다. 재시도할지, 대안을 요청할지, 아니면 작업을 그만둘지를 사용자가 직접 고르게 하면 된다. 이 세 가지를 모두 보여주면 사용자는 자신의 판단으로 다음을 결정할 수 있다. 특히 작업 포기는 가볍게 여겨지기 쉽지만 실제로 필요한 순간이 있다. 이 작업이 더는 의미가 없다고 판단했거나 시간이 없어 뒤로 미뤄야 한다고 판단했을 때, 사용자는 망설임 없이 포기를 선택할 수

있어야 한다.

3.2 실패 원인의 이해 가능한 설명

실패 메시지로 “오류가 발생했습니다”라는 표현을 쓰는 에이전트가 많다. 이 문장은 기술적으로는 정확하지만 사용자에게는 아무것도 주지 않는다. 사용자가 다음 행동을 결정하려면 왜 실패했는지 알아야 한다.

이해할 수 있는 설명에는 원칙이 세 가지 있다. 먼저 공정 수준에서 원인을 설명해야 한다. “네트워크 연결이 불안정해서 요청이 도달하지 못했습니다”라는 문장은 “요청 실패”보다 훨씬 많은 정보를 담는다. 다음으로 사용자가 다음에 뭘 할 수 있는지 암시해야 한다. “네트워크 연결을 확인한 후 다시 시도해 주세요”라는 조언이 있으면 사용자는 바로 행동할 수 있다. 마지막으로 기술적 용어는 피해야 한다. “TimeoutExceededException”이라는 예러 코드는 기술자에게는 유용해도 일반 사용자에게는 그저 소음일 뿐이다.

특히 반복 실패의 경우는 패턴을 보여주는 것이 효과적이다. “이미 3회 시도했지만 모두 네트워크 오류로 실패했습니다. 원인이 네트워크에 있을 가능성이 있습니다. 지금 잠시 후에 다시 시도하시는 것을 권장합니다”라는 설명은 단순한 실패 보고보다 훨씬 정보가 많다. 사용자는 시도 횟수를 알고 같은 이유로 계속 실패할 것 같다는 예측을 세우며 대기 시간을 조율할 수 있다.

3.3 재시도 경로와 대안 제안

단순한 “다시 시도”는 부족하다. 왜 다시 시도하면 성공할 것 같은지, 얼마나 기다려야 하는지, 그래도 안 되면 어떻게 해야 하는지를 함께 전달하면 사용자는 불안 없이 재시도할 수 있다.

재시도 경로는 세 단계로 구성하는 것이 좋다. 즉시 재시도, 잠시 후

재시도, 대안 제시 순이다. 즉시 재시도가 맞는 경우는 네트워크의 순간적 불안정이나 API의 일시적인 한도 초과처럼 오류가 잠깐 스치고 지나가는 상황이다. 잠시 후 재시도가 맞는 경우는 API의 rate limit이나 일시적인 서비스 중단처럼 시간이 지나야 풀리는 상황이다. 대안이 필요한 경우는 권한이 없거나 입력 데이터의 형식이 맞지 않는 것처럼 구조적인 문제가 원인일 때다.

대안은 반드시 구체적이어야 한다. “다른 방법을 사용해 보세요”라는 표현은 막연하다. 대신 “파일을 CSV 형식으로 변환 후 다시 업로드해 주시거나, 다른 파일을 선택해 주세요”라는 표현은 구체적인 행동을 제시한다. 에이전트가 대안을 복수 제시하면 사용자는 자신의 상황에 맞는 것을 선택할 수 있다.

3.4 치명적 실패와 일시적 실패의 구분

모든 실패가 같은 무게는 아니다. 데이터를 잃은 경우와 단순한 네트워크 타임아웃은 완전히 다른 차원이다. 사용자에게 어떻게 전달하고 무엇을 기대하게 할지는 실패의 종류에 따라 달라진다.

치명적 실패는 재시도로 해결되지 않고 사용자 개입이 반드시 필요한 경우다. 예를 들어 사용자의 권한이 부족한 경우, 제공된 입력 데이터가 요구사항을 충족하지 않는 경우, API 키가 유효하지 않은 경우 등이 해당한다. 이 경우 에이전트는 즉시 사용자에게 문제를 보고하고 구체적인 해결책을 제시해야 한다. “이 파일은 읽기 전용입니다. 쓰기 권한을 얻으려면 파일의 공유 설정을 변경해 주세요”라는 표현은 명확한 해결책을 제시한다.

일시적 실패는 재시도만으로 해결될 가능성이 높은 경우다. 네트워크 불안정, API rate limit, 서버의 일시적 과부하 등이 여기에 해당한다. 이 경우 에이전트는 사용자를 불안하게 하지 않는 것이 중요하다. “지금은 서버가 분주합니다. 잠시 후 다시 시도해 주세요”라는 표현은 일시적

문제임을 암시하면서도 문제가 중대하지 않다는 것을 전달한다.

실패의 중대성을 판단하는 기준을 에이전트가 먼저 적용하고 그에 맞는 수준의 메시지를 전달하는 것이 핵심이다. 치명적 실패를 일시적 실패처럼 처리하면 사용자가 불필요하게 재시도를 반복하며 시간을 낭비한다. 반대로 일시적 실패를 치명적 실패처럼 처리하면 사용자가 불필요하게 걱정하게 된다.

3.5 이 장의 핵심 정리

실패는 에이전트 제품에서 피할 수 없는 부분이다. 그러나 실패 이후의 대응 설계가 제품을 갈라놓는다. 부분 완료 상태를 시각적으로 유지하며 사용자에게 제어권을 돌려주고, 실패 원인을 공정 수준에서 설명하며, 재시도 경로와 대안을 단계적으로 제시하고, 실패의 중대성을 먼저 판단해 그에 맞게 대응하는 설계가 핵심이다. 이 원칙들을 충족하면 에이전트가 실패하더라도 사용자는 불안해하지 않고 다음에 할 올바른 행동을 취할 수 있다.

4 적응적 피드백 루프: 사용자와 에이전트의 공진화

에이전트의 최종 형태는 처음부터 완성되지 않는다. 사용자는 에이전트를 쓰면서 조금씩 손을 보고, 에이전트는 그 조정에 맞춰가면서 둘은 함께 진화한다. 이 장에서는 이 공진화 사이클을 어떻게 설계할지 다룬다.

4.1 수정의 명시적 반영

사용자가 에이전트의 결과물을 수정했을 때 에이전트가 그것을 감지해 다음에 활용하는지가 “학습하는 에이전트”와 “그렇지 않은 에이전트”를 가른다. 문제는 대다수 에이전트가 사용자의 수정을 전혀 알아채지 못한다는 점이다.

수정을 반영하는 가장 단순한 방식은 변경 사후 확인이다. 사용자가 결과물을 수정한 뒤 에이전트에 다시 메시지를 보내면 에이전트가 “혹시 이전 결과를 수정하셨나요? 그럼 변경점을 반영해서 다시 처리해 드릴까요?”라고 묻는다. 이 간단한 확인이 에이전트에게 학습 기회를 주는 동시에 사용자에게는 자신의 피드백이 의미 있었다는 확신을 준다.

더 발전된 방식은 적용 결과를 즉시 보여주는 것이다. 사용자가 “이런 식으로 해주세요”라고 수정을 요청하면 에이전트는 이를 즉시 적용해 결과를 보여주고, 동시에 “변경점을 반영해서 다시 처리했습니다. 맞으시면 계속 진행하시고 추가 수정이 필요하시면 말씀해 주세요”라고 확인을 구한다. 여기서 핵심은 “맞습니까?”라는 질문을 반드시 넣는 것이다. 이 질문이 있어야 사용자는 자신의 수정이 의도대로 반영되었는지 확인할 수 있다.

4.2 빈번한 작은 신호의 가치

피드백은 크지 않아도 된다. 오히려 작은 피드백이 더 값지다. “좋아요”나 “이건 별로야” 같은 단순한 반응도 쌓이면 에이전트에게 의미 있는 신호가 된다.

작은 신호를 포착하는 설계는 사용자의 암묵적 피드백을 잡아내는 것이다. 에이전트가 제안을 5개 했는데 사용자가 3개만 골랐다면 그 자체가 피드백이다. 에이전트는 “5개 중 3개를 선택하셨네요. 다음에는 3번과 5번 같은 유형의 제안을 더 드리겠습니다”라고 분석 결과를 전할 수 있다. 이 피드백은 사용자에게 자신의 행동이 기록되고 있음을 알려주는 동시에 에이전트가 그것을 실제로 활용한다는 것을 보여준다.

간단한 만족도 버튼도 효과적이다. 결과물 하단에 “만족스러우신가요?”와 함께 “네” “보통” “아니오” 버튼을 두면 사용자는 따로 글을 쓰지 않고도 만족도를 전할 수 있다. 이 데이터가 쌓이면 에이전트는 자신의 성능을 객관적으로 평가할 수 있다.

4.3 선호 기억의 경계와 관리

사용자의 선호를 기억해 두면 편리하지만 거기에는 경계가 있다. 무엇을 기억하고 무엇을 기억하지 않을지는 에이전트가 명시적으로 관리해야 한다.

기억해야 할 것은 문체 선호, 포맷 선호, 콘텐츠 주제 선호 같은 것이다. “항상 핵심만 요약해서 보여주세요”, “표보다는 불릿으로 표현해 주세요”, “결론을 먼저 보여주세요” 같은 선호가 이후 상호작용에서 계속 반영되면 사용자는 자신의 피드백이 의미 있었다고 느낀다.

기억하지 않아야 할 것은 보안에 민감한 정보, 일회성 작업, 부정적인 감정 표현이다. “비밀번호를 변경해 줘”라는 작업 후 에이전트가 비밀번호를 기억해서는 안 된다. 한 번만 수행하는 특이한 작업의 맥락을 선호로 오해해

반복 적용해서도 안 된다. 사용자가 불만을 표현했더라도 그것을 개인적인 감정으로 받아들이지 않고 시스템적 문제로 받아들여야 한다.

기억의 경계를 관리하는 설계는 명시적인 선호 설정 UI를 제공하는 것이다. 에이전트가 “이제까지의 선호 설정을 확인하시겠습니까?”라고 묻고 화면에 기억된 선호 목록과 함께 “이 항목을 삭제하시겠습니까?”라는 선택지를 주면 사용자는 자신의 데이터를 직접 관리할 수 있다. 이는 GDPR 같은 규제 대응에도 도움이 되면서 사용자에게 통제감을 준다.

4.4 자기 평가와 사용자 평가 사이의 간극

에이전트가 자신을 평가하는 방식과 사용자가 에이전트를 평가하는 방식 사이에는 근본적인 차이가 있는 경우가 많다. 에이전트는 스스로 세운 목표와 비교해 성공과 실패를 판단하지만 사용자는 자신의 기대와 결과물을 비교한다. 이 간극을 인식하지 못하면 에이전트는 성공했다고 여기는데 사용자는 불만족스러운 상황에 놓일 수 있다.

간극을 메우는 가장 효과적인 방식은 사전에 기대를 물어보는 것이다. 작업 시작 전 “어떤 결과물을 기대하고 계신가요?”라고 묻고 그 대답을 기억해서 결과물과 비교하는 방식이다. “시작하시기 전 기대하신 결과물은 '핵심 세 가지 포인트 요약'이었는데, 결과물이 그에 맞게 작성되었는지 확인해주세요”라는 메시지는 기대와 결과의 연결고리를 명확히 해준다.

에이전트의 자기 평가는 사용자 평가보다 늘 높게 나오는 경향이 있다. 에이전트가 자신의 과정을 기준으로 판단하기 때문이다. 그래서 에이전트가 자신 있는 결과물이라도 사용자에게 만족도를 물어보는 것이 유익하다. “제 기준으로는 적절히 처리된 것으로 판단하지만 혹시 불만족스러운 부분이 있으시면 말씀해 주세요”라는 겸손한 자세는 사용자에게 신뢰를 준다.

4.5 이 장의 핵심 정리

피드백 루프는 에이전트가 단순한 도구를 넘어 공동 진화하는 파트너로 나아가는 과정이다. 사용자의 수정을 명시적으로 반영하고 작은 피드백도 가볍게 넘기지 않으며 기억의 경계를 관리하면서 선호를 쌓고 자기 평가와 사용자 평가 사이의 간극을 인식해 다리를 놓는 설계가 핵심이다. 이 원칙을 지키면 에이전트는 시간이 갈수록 사용자의 만족도를 높여가고 사용자는 에이전트에게 피드백을 주는 일이 가치 있다고 느끼게 된다.